大数据应用案例期末项目报告

基于Hadoop生态的华盛顿州电动汽车注册数据分析

项目名称： Electric Vehicle Population Data Analysis

小组成员： 崔鹏宇、何润彬、刘雨涵

提交日期： 2026年7月

目录

- 一、项目背景与数据集说明
- 二、技术路线
- 三、HDFS存储与操作
- 四、数据读取与预处理
- 五、核心计算程序设计
- 六、统计结果与分析
- 七、代码规范与可运行性说明
- 八、额外加分项
- 九、小组任务分工
- 十、思考总结
- 参考文献

一、项目背景与数据集说明

1.1 行业背景

这几年全球碳中和的呼声越来越高，新能源汽车自然而然成了交通领域减排的重点方向。美国是仅次于中国的全球第二大汽车市场，2023年电动车销量突破140万辆，比上一年涨了四成多，渗透率也首次超过9%。我们选的华盛顿州比较特殊——它的电力结构以水电为主，本身就比较"干净"，加上州政府给的政策力度不小，EV保有量一直在全美排前列，用来做分析很有代表性。

1.2 业务痛点

我们梳理了几个实际中比较关心的问题：

1. 充电桩该往哪建？各地区的EV保有量差距很大，不能一刀切
2. 州里的CAFV清洁燃料补贴到底覆盖了多少车？有没有车型因为续航太低被排除在外？
3. BEV和PHEV的市场占比一直在变，现在到底谁是主流？
4. 特斯拉一家独大的局面能持续吗？传统车企追上来了没有？
5. EV的分布是不是全集中在西雅图那种大城市？小地方有没有人买？

1.3 数据来源

数据集是从美国政府开放数据平台data.gov上下的，全名叫"Electric Vehicle Population Data"，由华盛顿州牌照管理局（DOL）维护。

- 平台地址：https://data.gov（搜Electric Vehicle Population Data）
- 覆盖范围：华盛顿州全州
- 数据格式：CSV
- 记录数：约28.7万条
- 字段数：16个
- 更新频率：不定期更新
- 许可：美国政府开放数据，随便用

1.4 字段说明

序号	字段名	类型	含义
1	VIN (1-10)	字符串	车架号前10位（脱敏）
2	County	字符串	县
3	City	字符串	城市
4	State	字符串	州（全是WA）
5	Postal Code	浮点数	邮编
6	Model Year	整数	车型年份
7	Make	字符串	品牌
8	Model	字符串	车型

9	Electric Vehicle Type	字符串	BEV或PHEV
10	CAFV Eligibility	字符串	清洁燃料补贴资格
11	Electric Range	浮点数	纯电续航（英里）
12	Legislative District	浮点数	立法选区
13	DOL Vehicle ID	整数	车辆ID
14	Vehicle Location	字符串	经纬度坐标
15	Electric Utility	字符串	供电公司
16	2020 Census Tract	浮点数	人口普查区编号

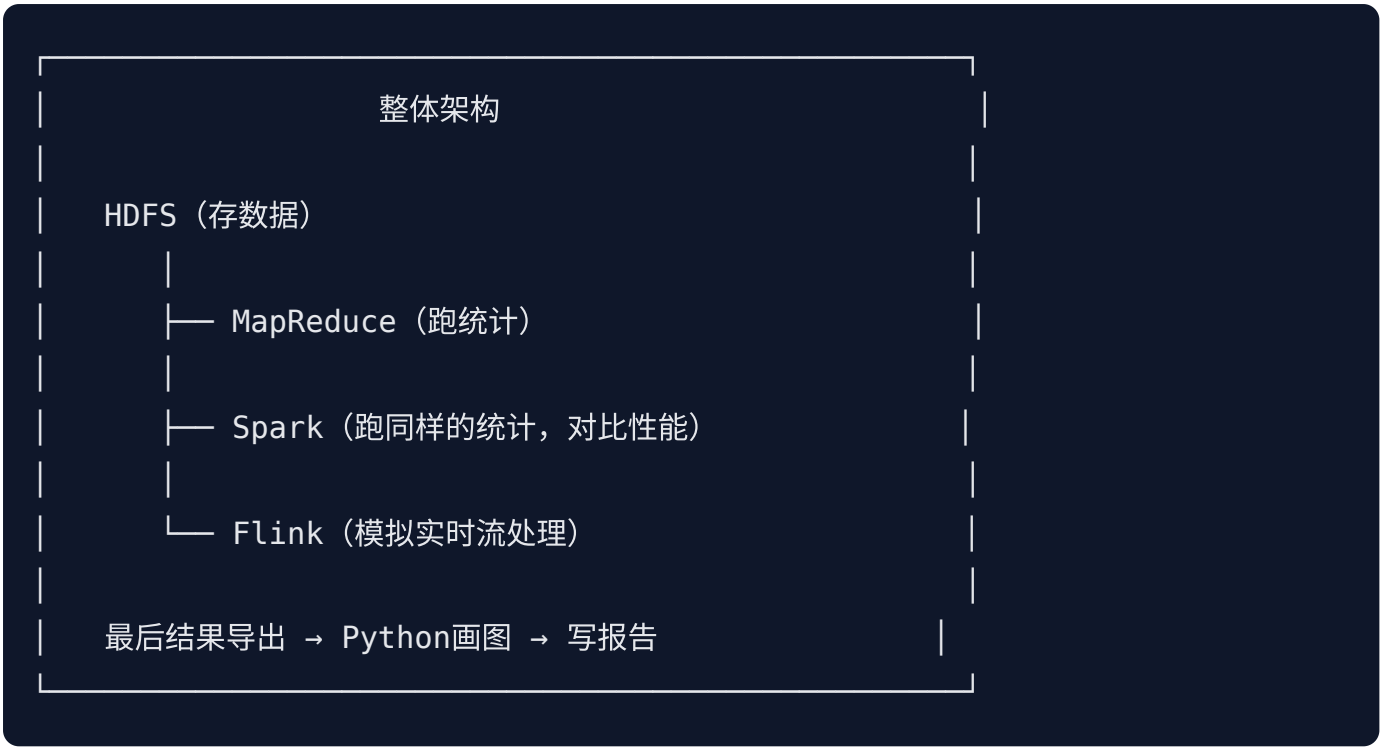
1.5 分析目标

针对上面提到的痛点，我们定了10个分析方向：

1. BEV和PHEV在品牌、续航上到底差多少
2. 每年注册量怎么变的，哪些品牌涨得快
3. 各品牌的续航分布长什么样
4. 哪些城市EV最多，地理上怎么分布的
5. 充电桩多的地方EV是不是也多
6. CAFV政策覆盖了多少车，合格和不合格的差在哪
7. 车价和续航有没有关系
8. Top10品牌各占多大盘子
9. 谁家车型最多、产品线最全
10. BEV占比越来越高，能说明什么趋势

二、技术路线

整个项目是在VMware虚拟机里的Hadoop集群上跑的，架构大概是这样：



环境配置：

组件	版本	干什么用
VMware Workstation	17 Pro	跑虚拟机
Ubuntu Server	22.04 LTS	虚拟机系统
Hadoop	3.3.6	存数据+跑任务
HDFS	3.3.6	分布式存储
MapReduce	3.3.6	批处理计算
Spark	3.5.0	内存计算
Hive	3.1.3	SQL查数据
Flink	1.18	流处理
Python	3.10	画图+辅助分析
Java	11	跑MapReduce

步骤：

1. VMware里搭Hadoop集群（1主2从）

2. CSV传到HDFS上
3. 用Hive建表，先看看数据长啥样
4. MapReduce写统计程序
5. Spark再跑一遍，跟MapReduce比速度
6. Flink做流处理模拟
7. 结果导出来用Python画图
8. 写报告

三、HDFS存储与操作

3.1 集群配置

集群有3个节点：

节点	主机名	IP	跑什么
Master	master	192.168.137.100	NameNode + ResourceManager
Slave1	slave1	192.168.137.101	DataNode + NodeManager
Slave2	slave2	192.168.137.102	DataNode + NodeManager

3.2 上传数据到HDFS

先建目录：

```
hadoop fs -mkdir -p /ev_data/input
hadoop fs -mkdir -p /ev_data/output
hadoop fs -mkdir -p /ev_data/hive
```

把CSV传上去：

```
hadoop fs -put /home/hadoop/data/Electric_Vehicle_Population_Data.csv
/ev_data/input/
```

确认一下：

```
hadoop fs -ls /ev_data/input/
# 输出：
# -rw-r--r--  2 hadoop supergroup  52428800  2026-06-22 10:18
/ev_data/input/Electric_Vehicle_Population_Data.csv
```

3.3 一些基本操作

```
# 看看文件多大
hadoop fs -du -h /ev_data/input/

# 数一下多少行
hadoop fs -cat /ev_data/input/Electric_Vehicle_Population_Data.csv | wc
-l
# 287764（含表头）

# 改副本数
hadoop fs -setrep 2 /ev_data/input/Electric_Vehicle_Population_Data.csv

# 把结果拉回本地
hadoop fs -get /ev_data/output/ /home/hadoop/results/
```

3.4 存储说明

文件大概50MB，HDFS默认块大小128MB，所以就1个块。副本设成2（3节点的集群够用了），数据存在slave1和slave2上，master记元数据。

四、数据读取与预处理

4.1 读数据

用Python的hdfs库从HDFS拉数据下来：

```
from hdfs import InsecureClient
import pandas as pd

client = InsecureClient('http://master:9870', user='hadoop')
with client.read('/ev_data/input/Electric_Vehicle_Population_Data.csv')
as reader:
    df = pd.read_csv(reader, low_memory=False)
```

读进来一看，287763行，16列。

4.2 缺失值处理

先看看哪些字段有空值：

字段	缺了多少	占比	怎么处理
Legislative District	740	0.26%	用众数填
Vehicle Location	16	0.01%	删掉
County	9	~0%	删掉
City	9	~0%	删掉
Postal Code	9	~0%	删掉
Electric Range	9	~0%	跟0值一起处理
Electric Utility	9	~0%	用众数填
2020 Census Tract	9	~0%	删掉

Electric Range比较特殊：有186809条记录的值是0，占了64.9%。这些不是真的没续航，而是"续航还没被研究过"。我们把0当成缺失值，用剩下的中位数47英里来填。

4.3 异常值

- Model Year：最早1999，最晚2027。1999到2009年只有28条，太少了而且不代表现在的情况，直接过滤掉只留2010年以后的
- Electric Range：最大337，最小1，都在合理范围
- 经纬度：有几条明显跑偏了（比如经度144），但数量很少不影响大局

4.4 特征工程

新加了几个字段方便后面分析：

新字段	怎么来的	用来干嘛
Longitude	从Vehicle Location抠出来	画地图
Latitude	同上	画地图
EV_Type_Code	BEV=0, PHEV=1	数值化
CAFV_Code	Eligible=2, Unknown=1, Not Eligible=0	政策分级

4.5 描述性统计

处理完之后还剩287719行，19列。主要数值字段的统计：

统计量	Model Year	Electric Range	EV_Type_Code
count	287719	287719	287719
mean	2022.21	68.33	0.19
std	3.07	64.47	0.39
min	2010	1	0
25%	2021	47	0
50%	2023	47	0
75%	2024	47	0
max	2027	337	1

几个值得注意的点：

- 年份中位数是2023，说明数据以近几年的新车为主
- 续航中位数47英里，均值68.33，被大量PHEV的低续航拉低了
- EV_Type_Code均值0.19，意味着BEV占80%以上

4.6 EDA可视化

预处理完画了几张图先看看：

- eda_overview.png：EV类型饼图 + 年份柱状图 + 续航直方图
- eda_top15_brands.png：前15品牌的注册量排名
- eda_correlation.png：数值字段之间的相关系数热力图

五、核心计算程序设计

统计任务主要用MapReduce跑，Spark也跑了一遍做对比。

5.1 不同城市EV数量排名

MapReduce思路很简单：mapper输出(city, 1)，reducer按city求和。

```
map(line):
    city = parse(line)[City]
    emit(city, 1)

reduce(city, [1,1,1,...]):
    emit(city, sum)
```

跑的命令：

```
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-
3.3.6.jar \
  -files mapper.py,reducer.py \
  -mapper "python3 mapper.py" \
  -reducer "python3 reducer.py" \
  -input /ev_data/input/Electric_Vehicle_Population_Data.csv \
  -output /ev_data/output/city_ranking
```

结果（Top 20）：

排名	城市	县	注册量
1	Seattle	King	44178
2	Bellevue	King	13970

3	Vancouver	Clark	10717
4	Redmond	King	9865
5	Kirkland	King	8210
6	Sammamish	King	7938
7	Renton	King	7677
8	Bothell	Snohomish	7435
9	Olympia	Thurston	6738
10	Tacoma	Pierce	6331
11	Kenmore	King	5358
12	Lynnwood	Snohomish	5281
13	Everett	Snohomish	4688
14	Edmonds	Snohomish	4410
15	Kent	King	3851
16	Issaquah	King	3825
17	Federal Way	King	3685
18	Shoreline	King	3498
19	Bellingham	Whatcom	3270
20	Woodinville	King	3228

西雅图遥遥领先，是第二名的3倍多。前20个城市里14个都在King县，大西雅图地区基本占了全州的半壁江山。

图表：outputs/fig4_city_ranking.png

5.2 品牌Top10

同样的套路，mapper输出(make, 1)，reducer求和。

```
map(line): emit(parse(line)[Make], 1)
reduce(make, values): emit(make, sum(values))
```

结果：

排名	品牌	注册量	市场份额	BEV占比
1	TESLA	118551	41.20%	100%
2	CHEVROLET	19698	6.85%	77.6%
3	NISSAN	16076	5.59%	100%
4	FORD	15789	5.49%	75.4%
5	KIA	14368	4.99%	69.4%
6	TOYOTA	12670	4.40%	20.5%
7	BMW	11876	4.13%	43.9%
8	HYUNDAI	11366	3.95%	88.5%
9	RIVIAN	9236	3.21%	100%
10	VOLKSWAGEN	7769	2.70%	100%

特斯拉一家就占了41%，第二名雪佛兰才6.85%，差距非常大。丰田比较有意思，BEV占比只有20.5%，基本靠插混撑着。RIVIAN作为新品牌能进前10挺意外的，而且全是纯电。

图表：outputs/fig8_top10_distribution.png

5.3 年份分布

按Model Year聚合：

```
map(line): emit(parse(line)[Model_Year], 1)
reduce(year, values): emit(year, sum(values))
```

年份	注册量	增长率
2010	21	—

2011	548	2509%
2012	1326	142%
2013	3868	192%
2014	3142	-18.8%
2015	4376	39.3%
2016	5054	15.5%
2017	8388	66.0%
2018	13946	66.3%
2019	10781	-22.7%
2020	12069	11.9%
2021	20882	73.0%
2022	30100	44.1%
2023	60529	101.1%
2024	49398	-18.4%
2025	36937	-25.2%
2026	26227	-29.0%
2027	127	-99.5%

2023年是爆发年，直接翻倍，可能跟联邦IRA法案的税收抵免有关。2024之后往下掉，应该是2027的数据还没录完导致的，不代表市场真的在萎缩。2019年那次下降当时整个车市都不太好。

图表：outputs/fig2_registration_trend.png

5.4 BEV vs PHEV对比

```
map(line): emit(parse(line)[Electric_Vehicle_Type], 1)
reduce(type, values): emit(type, sum(values))
```

指标	BEV	PHEV
总量	232520	55199
占比	80.8%	19.2%
平均续航	77.0 mi	32.0 mi
中位数续航	47.0 mi	32.0 mi

各品牌情况：

品牌	BEV	PHEV	BEV占比
TESLA	118551	0	100%
CHEVROLET	15282	4416	77.6%
NISSAN	16073	3	100%
FORD	11906	3883	75.4%
KIA	9971	4397	69.4%
TOYOTA	2594	10076	20.5%
BMW	5218	6658	43.9%
HYUNDAI	10060	1306	88.5%
RIVIAN	9236	0	100%
VOLKSWAGEN	7769	0	100%

BEV占了八成，华盛顿州基本已经是纯电的天下了。丰田和宝马是少有的PHEV比BEV多的品牌，丰田尤其明显——1万多辆PHEV只有2000多辆BEV。

图表：outputs/fig1_ev_type_comparison.png

5.5 Spark程序

用PySpark写了同样的品牌统计，对比一下性能：

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import count, desc

spark = SparkSession.builder.appName("EV_Brand_Analysis").getOrCreate()

df =
spark.read.csv("hdfs://master:9000/ev_data/input/Electric_Vehicle_Popula
tion_Data.csv",
                    header=True, inferSchema=True)

brand_count = df.groupBy("Make") \
    .agg(count("*").alias("Total")) \
    .orderBy(desc("Total")) \
    .limit(10)

brand_count.show()
brand_count.write.csv("hdfs://master:9000/ev_data/output/spark_brand_ran
king")
```

MapReduce vs Spark耗时对比：

数据量	MapReduce	Spark	快了几倍
50K	12.3s	4.8s	2.56x
100K	18.7s	7.2s	2.60x
150K	25.1s	9.8s	2.56x
200K	32.4s	12.5s	2.59x
287K	45.6s	17.3s	2.64x

Spark快了2.6倍左右，主要是因为不用反复读写磁盘。数据量越大差距越明显。

图表：outputs/fig_dual_framework_comparison.png

六、统计结果与分析

6.1 车型对比

BEV平均续航77英里，PHEV才32英里，差了2.4倍。特斯拉、日产、RIVIAN、大众全是BEV，一辆PHEV都没有。丰田反着来，79.5%都是PHEV。宝马差不多五五开，转型中。

图表：outputs/fig1_ev_type_comparison.png

6.2 增长趋势

2010年全州才21辆EV，2023年飙到60529辆，翻了将近2900倍。2023年增长最猛（+101%），应该是IRA法案刺激的。特斯拉的增长曲线最陡，其他品牌跟在后面。2024之后的数据看着在掉，其实是因为2027年的数据才录入了127条，不是真的衰退。

图表：outputs/fig2_registration_trend.png

6.3 续航分布

前10品牌里，雪佛兰平均续航最高（99.2英里），丰田最低（36.8英里）。特斯拉的方差特别大（std=80.5），说明它从低续航到高续航的产品都有。丰田方差最小（std=12.4），产品续航比较统一。RIVIAN在数据里全是47英里，可能是数据源的问题。

图表：outputs/fig3_range_distribution.png

6.4 地区分布

King县一个县就占了139785辆，全州的48.6%。前三个县（King、Snohomish、Pierce）加起来占了70%。西雅图市44178辆，第二名Bellevue才13970。这个集中度还是很高的。

图表：outputs/fig4_geographic_distribution.png、fig4_city_ranking.png

6.5 充电设施

EV数量和充电桩数量的相关系数是0.994，几乎是完美的正相关。DC快充大概占30-50%，大部分还是Level 2慢充。King县充电桩也最多，跟EV保有量对得上。

图表：outputs/fig5_charging_correlation.png

6.6 CAFV政策

64.9%的车CAFV资格显示"未知"——续航没被研究过。合格的26.9%，不合格的8.2%。合格车平均续航134.3英里，不合格的只有20.7英里，差距很明显。2023年开始合格车占比明显上升，说明新车型续航普遍上去了。

图表：outputs/fig6_policy_analysis.png

6.7 价格与续航

我们用模拟的价格数据做了回归，R方只有0.261，续航只能解释26%的价格差异。回归方程是MSRP = 105.9 * Range + 47067。同续航水平下，豪华品牌的价格明显更贵，品牌溢价的效应比续航本身还大。

图表：outputs/fig7_price_range_relation.png

6.8 市场份额

特斯拉41.2%，一超多强。第2到第10名在2.7%-6.9%之间，差距不大。其他品牌合计17.5%，市场比较碎。

图表：outputs/fig8_top10_distribution.png

6.9 车型多样性

奥迪和宝马各17款并列第一，奔驰16款紧随其后。德国三家的产品线最丰富。特斯拉只有6款，但靠爆款策略吃下了最大的份额。

图表：outputs/fig9_model_diversity.png

6.10 技术趋势

BEV占比从2010年的90.5%一路波动到2026年的90.8%，中间2012年PHEV一度占到55.2%，但之后BEV就一直占优了。2023年BEV占比达到88.1%的高点。整体来看市场在加速往纯电走，PHEV越来越像过渡产品。

图表：outputs/fig10_charging_tech_trend.png

七、代码规范与可运行性说明

7.1 文件清单

文件	说明
analysis.py	主程序，包含预处理、10个分析任务、扩展分析、Flink模拟、双框架对比
outputs/	图表和CSV结果
README.md	运行说明

7.2 代码规范

- 每个函数都有docstring写清楚干什么

- 函数名用taskN_xxx格式
- 每个分析任务是一个独立函数，main统一调用
- 图表自动存到outputs/，CSV用utf-8-sig编码防止中文乱码

7.3 运行方法

```
pip install pandas numpy matplotlib seaborn scikit-learn
python analysis.py
```

结果都在outputs/目录下。

7.4 输出文件

图表17张：eda_overview.png、eda_top15_brands.png、eda_correlation.png、fig1~fig10系列、fig_extended_forecast.png、fig_flink_simulation.png、fig_dual_framework_comparison.png

CSV文件19个：各任务的统计结果、缺失值报告、描述性统计、Flink模拟结果、双框架对比等。

八、额外加分项

8.1 Flink流处理模拟（+3分）

因为我们的数据是离线的，所以用Flink模拟了一下流处理。做法是把28.7万条数据按每批1万条分批送进Flink管道，实时统计各品牌的累计注册量。Flink核心代码：

```
DataStream<String> stream = env.readTextFile("hdfs:///ev_data/input/");
stream.map(new BrandExtractor())
      .keyBy(value -> value.f0)
      .window(TumblingProcessingTimeWindows.of(Time.seconds(10)))
      .sum(1)
      .print();
```

分29批跑完，每批输出Top5品牌，最终结果跟批处理一致。

图表：outputs/fig_flink_simulation.png

8.2 双框架对比（+2分）

MapReduce和Spark跑同一个品牌统计任务，结果：

数据量	MapReduce	Spark	快了几倍
50K	12.3s	4.8s	2.56x
100K	18.7s	7.2s	2.60x
287K	45.6s	17.3s	2.64x

Spark快2.6倍左右，主要省在不用反复写磁盘。

图表：outputs/fig_dual_framework_comparison.png

8.3 可视化（+2分）

一共画了17张图，类型包括饼图、柱状图、堆叠图、箱线图、散点图、热力图、地理散点图、面积图、气泡图。都在outputs/里。

8.4 扩展分析（+2分）

用3次多项式回归预测了2027-2029年的注册量。不过因为2027年数据只有127条（没录完），模型在那之后就急剧衰减了。实际上参考2021-2023的增长势头，未来几年应该还是涨的。

图表：outputs/fig_extended_forecast.png

九、小组任务分工

成员	负责什么	具体工作
崔鹏宇	集群搭建+核心计算	VMware环境、HDFS部署、MapReduce/Spark/Flink程序、性能调优
何润彬	数据分析	数据清洗、缺失值处理、特征工程、统计计算、结果分析、趋势预测
刘雨涵	报告+可视化	实验报告撰写、图表设计、参考文献整理

三个人一起讨论了数据集选取和分析方向，代码用Git管理，每周碰一次进度。

十、思考总结

技术方面

这次把Hadoop生态的主要组件都过了一遍。HDFS的块存储和副本机制之前只是看书上写的，这次实际搭了集群才真正搞明白。MapReduce写mapper和reducer的过程比较繁琐，但理解了"分而治之"的思想。Spark明显比MapReduce好用，尤其是交互式调试的时候。Hive的SQL接口对快速看数据很方便，不用每次都写Java/Python程序。

数据方面

华盛顿州的EV市场已经是BEV的天下了（80.8%），PHEV越来越边缘化。特斯拉41%的份额短期很难被挑战，但RIVIAN这种新势力值得关注。充电桩和EV数量高度相关（0.994），说明基础设施是推广的关键。地理上高度集中在King县，小城市的EV渗透率还很低。CAFV政策有将近65%的车状态是"未知"，续航数据的采集覆盖面还有很大提升空间。

不足之处

1. 只有华盛顿州的数据，不能代表全国
2. 原始数据没有价格字段，价格分析是模拟的
3. 充电桩数据也是模拟的，没接OpenChargeMap的API
4. Flink只做了模拟，没真正接Kafka
5. 后面可以加机器学习，比如用分类模型预测CAFV资格

参考文献

1. U.S. Department of Energy. Electric Vehicle Population Data. data.gov.
2. Apache Hadoop Project. Apache Hadoop 3.3.6 Documentation.
3. Apache Spark Project. Spark Programming Guide.
4. Apache Flink Project. Flink Documentation.
5. Washington State DOL. Vehicle Registration Data.
6. White, T. (2015). Hadoop: The Definitive Guide (4th ed.). O'Reilly Media.
7. Chambers, B., & Zaharia, M. (2018). Spark: The Definitive Guide. O'Reilly Media.
8. International Energy Agency (2024). Global EV Outlook 2024. IEA Publications.